

Computer Graphics: Lecture 9

The Matrix

Slide Deck © Grant Williams, 2026, License: [CC-BY-SA 4.0](#)

Welcome back!

Last time we reviewed the math we all forgot:

- We looked at the trig functions...
- ...and their inverses.
- The pythagorean theorem
- The dot product
- The cross product
- We saw some examples of when those things are handy in actual games or simulations.

This time

This is the most important mathematical lesson we're going to have.

We're going to learn about matrices.

But we're not just going to review: we're going to show how they enable 3D operations.

The matrix

So, okay. A matrix is a grid of numbers. Here's one:

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix}$$

But what does a matrix actually represent, and what does the number of rows and columns tell us about it?

Matrices are linear transformations

A matrix represents a function from vectors to vectors.

Specifically, they encode a linear function, where each component of the output is a **linear combination** of inputs.

What's a **linear combination**? It's a weighted sum.

So if we apply the function, each of the output values is treated as a weighted sum of the input values.

The rows of the matrix supply the weights.

Example matrix

Consider this simple 2 by 2 matrix: $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$

This matrix encodes a function: $f(v) = v'$, where $v'.x = v.x + 2*v.y$ and $v'.y = 3*v.x + 4*v.y$.

Those are both weighted sums.

To *call* the function represented by a matrix, we post-multiply it by the column matrix we want to pass as input.

Applying a matrix

For example, let's call that matrix on the vector $\langle 5, 6 \rangle$

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 \\ 6 \end{pmatrix} = \begin{pmatrix} 5 \cdot 1 + 6 \cdot 2 \\ 5 \cdot 3 + 6 \cdot 4 \end{pmatrix} = \begin{pmatrix} 17 \\ 39 \end{pmatrix}$$

The columns provide a particularly good way to understand a matrix. Each column of a matrix is a **basis vector**. Each **basis vector** tells you how the matrix will transform the corresponding input component.

The basis vector $\langle 1, 3 \rangle$ above tells us that for every unit of x input, the output will move in the $\langle 1, 3 \rangle$ direction by one unit. So we moved 5 units in that direction, and 6 units in the $\langle 2, 4 \rangle$ direction.

Why bother?

If you've tried to learn 3D graphics before, you probably were given a list of matrices. It's near the beginning of any 3D curriculum.

Why? Because there are a lot of things we want to do to our models: stretch them, squeeze them, rotate them, move them around, look at them in a camera, project the camera into perspective.

All of these things can be done with matrices (assuming we're in a projective space, which I will talk about later).

Why bother? (2)

But more importantly: those things can all be done *in sequence* with a single matrix. One matrix can represent *every sequence of any one* of these actions.

We can stretch, squeeze, rotate, move, view, and project, in one single operation.

In fact, we can stretch three times, then move, then stretch again.

That sequence of actions will have *one* corresponding matrix.

Why bother? (3)

Therefore, it's wonderful for computer graphics: we don't need to submit a list of actions to the video card. A single matrix suffices.

Granted, there is more than one *kind* of thing that needs to be transformed. In practice, there will usually be several matrices that the video card sees:

- One for the model (stretching, squashing, moving, rotating)
- One for the camera (viewing)
- One for the projection (perspective, can be combined with camera)
- One for each animation bone (when using skeletal animations)
- Lights can use matrices, too.

Questions?

Matrix sizes

A matrix does not have to be square.

The number of columns of a matrix tells you how many components the input vector must have.

The number of rows tells you how many output components the matrix will have.

If a matrix has fewer rows than columns, the matrix is necessarily cutting away dimensions.

Likewise, if a matrix has more rows than columns, the matrix is necessarily adding dimensions.

Matrix sizes (2)

However, our matrices will be square. A square matrix results in the same number of components of output as input.

Since we're working in 3D, let's start with a 3x3 matrix and see how far that gets us.

Each matrix will represent an operation that we want to do, such as rotation. To use the matrix, we will multiply it by all the vertex positions in the model.

Assume the model will be centered, and that these operations happen around the origin.

The scaling matrix

One of the simplest matrices is the scaling matrix. It can be used to stretch and squeeze a 3D model when we apply it to the vertices.

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix}$$

This matrix will keep the x component the same. The y component will double, and the z component will triple.

So if we applied this matrix to $\langle 2, 2, 2 \rangle$ we would get $\langle 2, 4, 6 \rangle$

All the vertices of the model are pushed away (or pulled toward for values less than 1) zero.

The identity matrix

If we “scale” every component by 1, then nothing will change. This is the identity matrix. It is the matrix equivalent of 1. Multiply a compatible vector by it: nothing happens.

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

When is this one useful? It’s very useful for debugging: set the current model’s matrix to the identity and it should be hanging out the origin, looking the same as it looked in the 3D modelling program.

The rotation matrix

This is the first one that requires some explanation.

Let's start by figuring out how it works in 2D.

If we want to rotate a *point* in 2D. How do we do it?

E.g., let's say we have the point described by the vector $\langle 1, 1 \rangle$. How do we rotate that by, say, 10% of a turn around the origin?

Rotation in 2D

Let's break it down by component.

Imagine that we rotated just the x axis.

The new value of the axis was pointing at $(1, 0)$. Now it will point to $(\cos(\theta), \sin(\theta))$.

The new y axis is going to be rotated the same amount, but $25\%\tau$ forward: $(\cos(\theta + 25\%\tau), \sin(\theta + 25\%\tau))$

This results in $-\sin(\theta), \cos(\theta)$ to be the new y-axis.

But what do we do with these new axes?

Rotation in 2D (2)

We take the old x value and multiply it by the new x-axis. That stretched x axis is part of the answer.

We take the old y value and multiply it by the new y-axis. That stretched y axis is the other part of our answer.

We add those two parts together.

I've just described matrix multiplication by this matrix:

$$\begin{pmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{pmatrix}$$

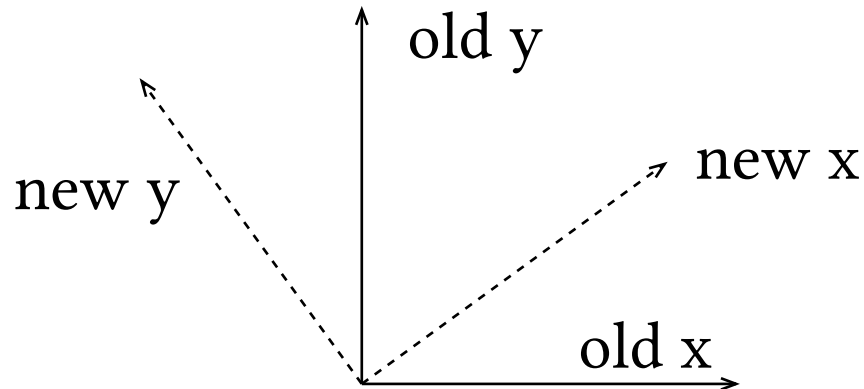
Rotation in 2D–basis vector version

But it's easier to understand if we think about it as basis vectors.

A 2x2 matrix is just two independent axes: one x, one y.

Rotate *both* of them by theta.

The new matrix just has the basis vectors pointing in the rotated direction:



Rotation about Z in 3D

It turns out, if we want to rotate within the X-Y plane like that, in three dimensions it's exactly the same, except we ignore the z axis. That is, the z axis stays the same:

$$\begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

The rotation matrix around the Z-axis.

But what if we want to rotate around the X or Y axes?

Rotation about X or Y in 3D

The axis we're rotating around doesn't change. It gets “passed through”.

Here is rotation around X:

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{pmatrix}$$

Here is rotation around Y¹:

$$\begin{pmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{pmatrix}$$

¹The signs on the sines are flipped because Z comes *out* of the screen, so the relationship between Z and X are opposite the relationship between X and Y.

Handedness of coordinate systems

Just a quick aside: I mentioned that the Z-coordinate comes *out* of the screen, and the relationship between Z and X is what caused the signs to be flipped on some of the coefficients.

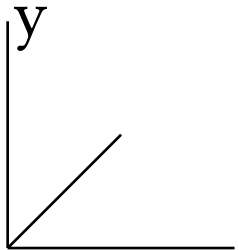
Z doesn't have to come out of the screen. If it does, we say that the coordinate system is **right-handed**.

If Z goes *into* the screen, the coordinate system is **left-handed**.

Handedness of coordinate systems (2)

Hold your left hand up and make an “L”. Poke your middle finger forward. Your middle finger is “+Z”, your thumb is “+X”, and your index finger is “+Y”.

Hold your right hand up with your thumb and index finger going the same way. You will have to twist it around. Z (your middle finger) will point toward you.



Left handed

Handedness of coordinate systems (3)

The depth buffer in WebGL is actually left-handed: larger Z values are considered behind smaller ones, so Z goes into the screen.

Affine matrices

Swizzling